ACADEMIQ (AI-DRIVEN LMS)

MAJOR PROJECT-II

(20SSP65)

Submitted by

PERUMAL P

21MSS034

Under the Guidance of

LT. Dr. D. ANTONY ARUL RAJ, M.Sc., M.Phil., Ph.D., PGDCE., NET.,

Associate Professor

Department of Software Systems

In partial fulfillment of the requirements for the award of the degree of

MASTER OF SCIENCE IN SOFTWARE SYSTEMS

(Five-Years Integrated Programme)

of Bharathiar University

DEPARTMENT OF SOFTWARE SYSTEMS

PSG COLLEGE OF ARTS & SCIENCE

An Autonomous College - Affiliated to Bharathiar University

Accredited with 'A^{++}'Grade by NAAC (4th Cycle)

College with Potential for Excellence

(Status Awarded by the UGC)

Star College Status Awarded by DBT - MST

An ISO 9001:2015 Certified Institution

Coimbatore - 641 014

April 2026

DEPARTMENT OF SOFTWARE SYSTEMS

PSG COLLEGE OF ARTS & SCIENCE

An Autonomous College - Affiliated to Bharathiar University

Accredited with 'A^{++}'Grade by NAAC (4th Cycle)

College with Potential for Excellence

(Status Awarded by the UGC)

Star College Status Awarded by DBT - MST

An ISO 9001:2015 Certified Institution

Coimbatore - 641 014

CERTIFICATE

This is to certify that this project work entitled **"AcademIQ (AI-Driven LMS)"** is a bonafide record of work done by **Perumal P (21MSS034)** in partial fulfillment of the requirements for the award of Degree of **Master of Science in Software Systems** (Five Years Integrated Course) of Bharathiar University.

_____ _____

Faculty Guide **Head of the Department**

Submitted for Viva-Voce Examination held on _____

_____ _____

Internal Examiner **External Examiner**

DECLARATION

I, **Perumal P (21MSS034)**, hereby declare that this Project work entitled **"AcademIQ (AI-Driven LMS)"**, is submitted to PSG College of Arts & Science (Autonomous), Coimbatore in partial fulfillment for the award of Master of Science in Software Systems, is a record of original work done by me under the supervision and guidance of **LT. Dr. D. Antony Arul Raj, M.Sc., M.Phil., Ph.D., PGDCE., NET.,** Associate Professor & Head of the Department, in Department of Software Systems, PSG College of Arts & Science, Coimbatore.

This report has not been submitted by me for the award of any other Degree/ Diploma/Associate ship/ Fellowship or any other similar degree to any other university

Place: Coimbatore **Perumal P**

Date: **21MSS034**

ACKNOWLEDGEMENT

My venture stands imperfect without dedicating my gratitude to a few people who have contributed a lot towards the victorious completion for my project work.

I would like to thank **Thiru L. Gopalakrishnan, Managing Trustee, PSG & Sons Charities**, for providing me a prospect and surroundings that made the work possible.

I take this opportunity to express my deep sense of gratitude to **Dr. T. Kannaian M.Tech., Ph.D.,** Secretary, PSG College of Arts & Science, Coimbatore for permitting and doing the needful towards the successful completion of this project.

I express my deep sense of gratitude and sincere thanks to our Principal **Dr. M. Jayanthi, M.Com., M.Phil., Ph.D.,** for her valuable advice and concern for students. I am very thankful to **Dr. M Umarani M.B.A., M.Phil., Ph.D.,** Vice Principal for their support towards my project.

I sincerely thank **Dr. K.V. Rukmani MCA., M.E., Ph.D., NET.,** Associate Professor and Head of the Department, Department of Software Systems for her wholehearted help to complete this project successfully by giving valuable suggestions.

I convey my heartiest and passionate sense of thankfulness to my project guide **LT. Dr. D. Antony Arul Raj, M.Sc., M.Phil., Ph.D., PGDCE., NET.,** Assistant Professor, Department of Software Systems, for his timely suggestion which enabled me in completing the project successfully.

I express my sincere gratitude to **Mr. Sajish Kumar Athra Pulikkal Mohanan,Assistant Manager - Web Development, GIBS - Digital - eCommerce Development, Avantor, Inc.**, for providing me with valuable guidance, continuous support, and expert insights throughout the development of this project.

This note of acknowledgment will be incomplete without paying my heartful devotion to my parents, my friends, and other people, for their blessings, encouragement, financial support, and the patience, without which it would have been impossible for me to complete the job.

Perumal P

21MSS034

INTERNSHIP OFFER LETTER

avantor™

VWR Lab Products Private Limited
Rathinam Techno Park, Pollachi Main Road,
Eachanari Post, Coimbatore - 641021
Regd Office
Unit No. 10, Ground Floor, Empire Infantry Building,
Plot No. 29, Infantry Road,
Bangalore 560001

September 19, 2025

Perumal Palaniappan
710, Vinayagar Kovil Street,
Lawly Road,
Coimbatore - 641003

Mobile: 8838375900
Email: perumal.palaniappan04@gmail.com
College/University: PSG College Of Arts & Science

Subject: Internship opportunity at VWR Lab Products Private Limited

Dear Perumal Palaniappan,

We write to you for and on behalf of VWR Lab Products Private Limited having its registered office at Unit No. 10, Ground Floor, Empire Infantry Building, Plot No. 29 (old Plot no. 10), Infantry Road, Bangalore-560001, Karnataka, India (hereinafter referred to as "**Company**"), with reference to your application for an internship at the Company.

We are pleased to offer you an internship opportunity with the Digital Engineering department based at Coimbatore. You will be reporting to Sajish Kumar Athra Pulikkal Mohanan (**"Reporting Manager"**). The Reporting Manager in consultation with you will develop a program description and expectations designed to achieve the objectives of the internship. The internship will provide you with valuable learning experience and is directly related to the subject matter of your studies as a full-time student at your College/University. Company's goal in developing this internship is to help you gain first-hand experience and for you to observe how your course of study is applied in a real-world setting and improve your skills in those areas.

Scope of Internship: Student Internship with the Digital Engineering Team.

Duration of the Internship: Six (6) months, i.e., from September 19, 2025 to March 19, 2026.
The duration of the internship may be extended at the sole discretion of the Company.

Stipend: During the duration of the internship you shall be paid an amount of INR 20,000/- per month (hereinafter referred to as "Stipend") for the expenses that you may incur during the duration of the internship. You shall not be entitled to receive any other payment from the Company during the internship, unless and until confirmed in writing by the Company. All taxes payable on the Stipend shall be borne by you and the Company shall have no liabilities in this respect. The Stipend so payable by the Company to you as provided above shall be subject to applicable income-tax deductions at source, if any. Payment of the stipend shall not be construed as creating any employment relationship between you and the Company.

You hereby confirm and acknowledge you are participating in the internship and that the internship will be conducted without entitlement to a paid job and/or any other form of employment with the Company at the conclusion of the internship.

The location of your internship will be the Coimbatore office of the Company. During the internship period, you will be required to attend office regularly on all working days. In case of any inability to attend office or if you need to take leave, you must inform your Reporting Manager in advance.

Perumal.P

INTERNSHIP COMPLETION LETTER

Rndsoft
An ISO Certified Company

September 03, 2024

TO WHOMSOEVER IT MAY CONCERN

This is to state that **Mr. PERUMAL.P,** Register No –**21MSS034**, 4th Year **MSC SOFTWARE SYSTEMS** Student from **"PSG COLLEGE OF ARTS AND SCIENCE"** has successfully completed his **SOFTWARE** Internship training from **JUNE 03, 2024** to **SEPTEMBER 03, 2024** in our company.

We wish him every success in life.

RND SOFTECH PRIVATE LIMITED Rndsoft COIMBATORE

For RND Softech Private Limited,

Asst. Manager – Talent Acquisition

RND SOFTECH PRIVATE LIMITED
S.F.No : 274/4, Anna Private Industrial Estate, Vilankurichi Road, Coimbatore - 641 035, India.
Tel : +91 85264 44408, 85264 44409, CIN : U72200TZ1998PTC008407, GSTIN : 33AABCR4136Q1Z5, IECN : 3899000099
Email : info@rndsoftech.com, Visit us at : www.rndsoftech.com

SYNOPSIS

AcademIQ is an AI-powered Learning Management System designed to modernize and streamline digital education through intelligent automation and a user-friendly interface. The system integrates various modules such as course management, content delivery, student tracking, and instructor tools into a single platform. It uses modern frontend technologies like React.js and Tailwind CSS to provide a responsive, fast, and seamless user experience suitable for both students and educators.

The platform incorporates secure authentication through Google and GitHub OAuth, along with a role-based access system that categorizes users as Students, Instructors, or SuperAdmins. Each role is provided with dedicated dashboards and permissions that align with its responsibilities, ensuring structured workflow and controlled access. The backend, powered by Node.js, Express.js, and PostgreSQL, manages data efficiently while maintaining scalability, security, and smooth API communication across the system.

A key highlight of AcademIQ is its integration with Cohere AI (Command-A), which enables intelligent features such as AI-assisted content generation, automated summary creation, and personalized learning support. This reduces the manual workload for instructors and enhances the learning experience for students. By combining a powerful technical stack with smart AI capabilities, AcademIQ delivers a next-generation LMS solution that is efficient, interactive, and adaptable to the evolving needs of digital education.

TABLE OF CONTENTS

CHAPTER NO	CONTENTS	PAGE NO
1	**INTRODUCTION**	
	1.1 Company Profile	1
	1.2 Project Overview	2
	1.3 Module Description	3
2	**SYSTEM ANALYSIS**	
	2.1 Existing System	5
	2.2 Proposed System	5
3	**SYSTEM CONFIGURATION**	
	3.1 Hardware Specification	6
	3.2 Software Specification	6
4	**SOFTWARE DESCRIPTION**	
	4.1 Node.js (Backend Runtime Environment)	7
	4.2 Express.js (Web Framework)	7
	4.3 React.js (Frontend Library)	7
	4.4 Tailwind CSS (Styling Framework)	8
	4.5 PostgreSQL (Database Management System)	8
	4.6 Cohere AI (Command-A Model – AI Integration)	8
5	**SYSTEM DESIGN**	
	5.1 Data Flow Diagram	9
	5.2 Entity Relationship Diagram	12
	5.3 Input Design	13
	5.4 Output Design	13
6	**SYSTEM IMPLEMENTATION & TESTING**	14
7	**CONCLUSION & FUTURE RECOMMENDATION**	17
	BIBLIOGRAPHY	19
	APPENDIX	
	Screenshots	20
	Sample Coding	24

CHAPTER 1

INTRODUCTION

1.1 COMPANY PROFILE

Avantor is a global life-science tools company that supplies mission-critical products, materials, and services to customers in biopharma, healthcare, education and government, and advanced technologies, operating through a worldwide manufacturing, distribution, and digital network. The VWR brand and the vwr.com channel form a core part of Avantor's go-to-market model, serving customers in over 180 countries and shipping to hundreds of thousands of locations across the globe. These operations are supported by a broad portfolio and a dedicated e-commerce ecosystem designed to streamline laboratory procurement and services.

In India, VWR functions as Avantor's channel for laboratory products and services, offering local commercial support and access to the global catalogue through region-specific portals and contact hubs. This integrated presence connects Indian customers with Avantor's worldwide supply chain and service standards, ensuring consistent access to laboratory consumables, equipment, chemicals, and related solutions.

The Coimbatore site is publicly referenced as a Global Business Center that supports VWR/Avantor's back-end and shared-services operations, located at Rathinam Techno Park, Eachanari, on Pollachi Main Road. Listings describing the campus identify its role in corporate services and provide local switchboard details, reflecting the site's position within Avantor's broader channel and shared-services strategy. These sources place the facility within the Eachanari/Coimbatore Industrial Estate area and highlight its growth as part of VWR's India footprint.

1.1.1 CONTACT

Company: VWR Lab Products Private Limited (VWR, part of Avantor) Coimbatore

Address: Rathinam Techno Park, Eachanari (PO), Pollachi Main Road, Coimbatore, Tamil Nadu 641021, India

Phone: +91 422 434 3300

1.2 PROJECT OVERVIEW

AcademIQ is an **AI-enhanced Learning Management System (LMS)** developed to transform digital education through automation, intelligent assistance, and a modern user-centric interface. The platform enables seamless management of courses, assessments, learning materials, and user activities across students, instructors, and administrative roles. Built with React.js and Tailwind CSS on the frontend and powered by Node.js, Express.js, and PostgreSQL on the backend, AcademIQ ensures a smooth, scalable, and secure environment for online learning. It supports OAuth-based authentication through Google and GitHub, offering a secure and frictionless login experience while enforcing role-based access to maintain structured control across the platform.

A highlight of AcademIQ is its integration of Cohere AI (Command-A) to introduce intelligent automation into the learning ecosystem. The AI engine assists instructors in generating course content, summaries, and learning materials, while students benefit from AI-driven explanations and study assistance. SuperAdmins gain full platform visibility with advanced dashboards and system management capabilities. By combining a robust tech stack with powerful AI features, AcademIQ delivers an innovative LMS solution that enhances productivity, enriches learning outcomes, and supports modern educational needs with efficiency and intelligence.

1.3 MODULE DESCRIPTION

- **Authentication & User Access Module**

This module handles secure login through traditional credentials as well as OAuth providers like Google and GitHub. Users select their role during registration, and the system automatically assigns permissions and redirects them to their respective dashboards. It ensures proper session handling, protected routes, and secure access control for all roles.

1.3.1 STUDENT MODULES

- **Student Dashboard Module**

The student dashboard displays personalized learning insights such as total courses, generated notes, flashcards, and quiz status. It provides quick access to AI tools and recently viewed courses, helping students resume their learning seamlessly. A visual activity graph highlights their weekly learning performance.

- **Explore Courses & Learning Module**

Students can browse and enroll in courses categorized by subject and difficulty. Each course provides structured modules, downloadable resources, and lesson-wise content. The module ensures smooth navigation through the entire learning journey.

- **AI Notes Generator Module**

Students can generate instant notes on any topic using Cohere Command-A by selecting difficulty and content preferences. The generated notes are neatly structured for learning and revision. Recent generations are stored for quick future reference.

- **AI Flashcards Generator Module**

This module allows students to create topic-based flashcards by selecting difficulty levels and card count. The AI generates concise, easy-to-remember flashcards to support faster revision. Students can save and reuse flashcards anytime.

- **Quiz Module**

Students can attempt quizzes assigned by instructors, categorized into Pending, Completed, and New. Each quiz displays the number of questions and instructor details before starting. After submission, students can view scores and track their quiz performance history.

1.3.2 INSTRUCTOR MODULES

- **Instructor Dashboard Module**

The instructor dashboard highlights metrics like total courses created, enrolled students, and quizzes assigned. A graphical overview shows how students interact with the instructor's content. Shortcut buttons allow quick creation of courses and quizzes.

- **Course Creation & Management Module**

Instructors can create structured courses with banners, objectives, and multi-module content. They can also specify special features like certificates or downloadable PDFs. All created courses can be edited, published, or managed from the "My Courses" section.

- **Quiz Creation & Management Module**

This module enables instructors to create quizzes with various question types such as MCQs and checkboxes. Instructors can set titles, instructions, and publish quizzes to students. They can also track quiz status (Pending/Accepted) and edit them when needed.

1.3.3 SUPERADMIN MODULES

- **SuperAdmin Dashboard Module**

The SuperAdmin dashboard provides platform-wide analytics such as total users, courses, quizzes, and recent registrations. Graphs visualize user growth and platform activity in real time. It also includes quick controls for managing users, courses, and quizzes.

- **User Management Module**

SuperAdmins can view all platform users, update their roles, or remove inactive accounts. This module ensures only authorized individuals have access to sensitive or creation-based features. It maintains the overall stability and integrity of the platform.

- **Course & Quiz Oversight Module**

SuperAdmins can review all courses and quizzes, ensuring quality and compliance with platform guidelines. They can adjust visibility, monitor instructor activity, and maintain content standards. This module ensures smooth academic workflow across the system.

1.3.3 GLOBAL MODULE

- **Settings & Account Management Module**

Users can update personal information, modify passwords, and switch between light and dark themes. The settings area centralizes all customization options for better user control. It also contains a "Danger Zone" that allows users to permanently delete their accounts.

CHAPTER 2

SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

The current online learning solutions rely heavily on manual content creation by instructors, which increases time consumption and reduces teaching efficiency. Many LMS platforms lack integrated AI features, making students depend on external tools for generating notes, quizzes, and study materials. Authentication systems in traditional LMS applications are outdated and often do not support OAuth logins such as Google or GitHub. Monitoring and management tools for administrators are limited, restricting visibility into user activity, course performance, and system-wide analytics. Students usually do not have access to interactive learning aids like flashcards, instant note generation, or AI-driven learning assistance. Existing platforms often fail to provide clean role-based dashboards, causing cluttered interfaces and difficulty navigating between modules.

2.2 PROPOSED SYSTEM

AcademIQ introduces an AI-powered LMS that automates learning processes such as note generation, flashcards, and quiz support using Cohere Command-A. The system integrates secure and modern authentication through Google and GitHub OAuth, ensuring seamless access and enhanced security. A role-based access control structure is implemented, offering dedicated dashboards for Students, Instructors, and SuperAdmins to streamline workflow. Instructors can efficiently create and manage courses and quizzes using an organized interface with AI assistance to reduce manual workload.

The platform offers advanced analytics and monitoring tools for SuperAdmins, enabling full control over users, courses, and quizzes. Students benefit from structured learning modules, AI tools, and progress tracking features that improve engagement and personalized learning.

CHAPTER 3

SYSTEM CONFIGURATION

3.1 HARDWARE SPECIFICATION

COMPONENTS	REQUIREMENTS
• Processor	: Intel Core i3 (10th Gen) / AMD Ryzen 3
• Hard Disk	: 256 GB SSD
• RAM	: 8 GB
• System Type	: 64-bit operating system, x64-based processor
• Keyboard	: Standard 104-key keyboard
• Mouse	: Optical Mouse

3.2 SOFTWARE SPECIFICATION

COMPONENTS	REQUIREMENTS
• Operating System	: Windows 10 / Windows 11
• Frontend Development Tools	: React.js, Tailwind CSS, NPM
• Backend Development Tools	: Node.js, Express.js
• Database	: PostgreSQL
• AI Integration	: Cohere AI (Command-A)
• Version Control	: Git & GitHub
• Execution Environment	: Visual Studio Code
• Browser	: Google Chrome / Microsoft Edge

CHAPTER 4

SOFTWARE DESCRIPTION

4.1 Node.js (Backend Runtime Environment)

Node.js is a high-performance JavaScript runtime built on Chrome's V8 engine, enabling fast and non-blocking backend server execution. It is widely used to develop REST APIs, handle asynchronous processes, and support scalable server-side logic. Node.js integrates seamlessly with npm (Node Package Manager), allowing developers to install thousands of libraries for authentication, routing, database connectivity, and more. Its event-driven architecture ensures efficient handling of multiple client requests.

4.2 Express.js (Web Framework)

Kaggle Notebook is an integrated development environment (IDE) that provides tools for data analysis and machine learning. It allows users to write and execute Python code in a browser-based environment, making it suitable for the accident severity prediction model. The platform supports various libraries and packages essential for data processing, analysis, and model building, facilitating a streamlined workflow for predictive analytics. Express.js is a lightweight and flexible Node.js framework used to build backend APIs and routing structures. It simplifies server creation, middleware management, and request handling, making development faster and more organized. In AcademIQ, Express.js powers secure role-based routing, authentication middleware, and integration with PostgreSQL and Cohere AI. Its modular structure enables maintainable and clean backend code.

4.3 React.js (Frontend Library)

React.js is a highly efficient JavaScript library for building dynamic user interfaces through reusable components. It provides fast rendering using the Virtual DOM, enabling smooth interactions in dashboards, forms, and course pages. React's component structure supports modular UI design, improving maintainability throughout AcademIQ. It integrates easily with REST APIs, authentication systems, and state management tools.

4.4 Tailwind CSS (Styling Framework)

Tailwind CSS is a utility-first CSS framework that enables rapid and responsive UI development. It eliminates the need for writing large CSS files by providing pre-built classes that can be applied directly within JSX components. Tailwind helps create clean, modern, and mobile-friendly designs throughout the LMS interface. It also supports full customization through configuration files for colors, spacing, typography, and themes.

4.5 PostgreSQL (Database Management System)

PostgreSQL is a powerful open-source relational database used to manage users, courses, quizzes, and platform-wide data. It supports ACID transactions, ensuring reliability and secure data storage for LMS applications. With strong SQL querying, indexing, and scalability features, PostgreSQL enables fast data access and efficient backend operations. It integrates seamlessly with Express.js through ORM tools or native drivers.

4.6 Cohere AI (Command-A Model – AI Integration)

Cohere's Command-A model powers intelligent educational tools such as notes generation, flashcards creation, and content summarization. It delivers accurate, context-aware responses based on the user's topic, difficulty preference, or study need. The API integrates with the backend to provide real-time AI outputs for both students and instructors. This enables automation of academic content, reducing workload and enhancing learning.

CHAPTER 5

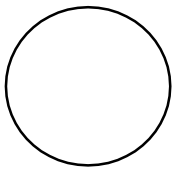
SYSTEM DESIGN

5.1 DATA FLOW DIAGRAM

A data flow diagram (DFD) is a graphical representation of the "flow" of data through an information system. DFD can also be used for the visualization of data processing. The purposes of DFD are as follows:

- To show the scope and boundaries of a system.
- To show that the whole system has been considered.
- May be used as a communication tool between a system analyst and any person who plays a part in the system.
- To act as the starting point for redesigning a system.

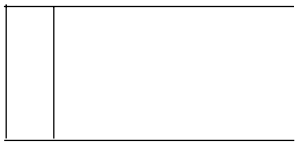
The representation used to frame a data flow diagram is:



- The circle represents the process. It shows the pair of processes that transform input to output.

- The arrow represents the graphical flow into or out of the process.

- The terminator represents external entities with systemcommunication.



- The store is used to model a collection of data packets at rest.

Level 0:

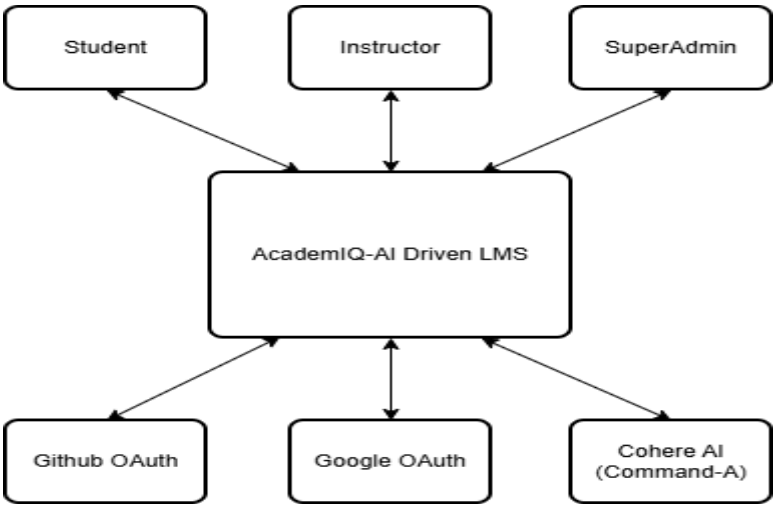


Figure 5.1.1 Level 0 DFD

Level 1:

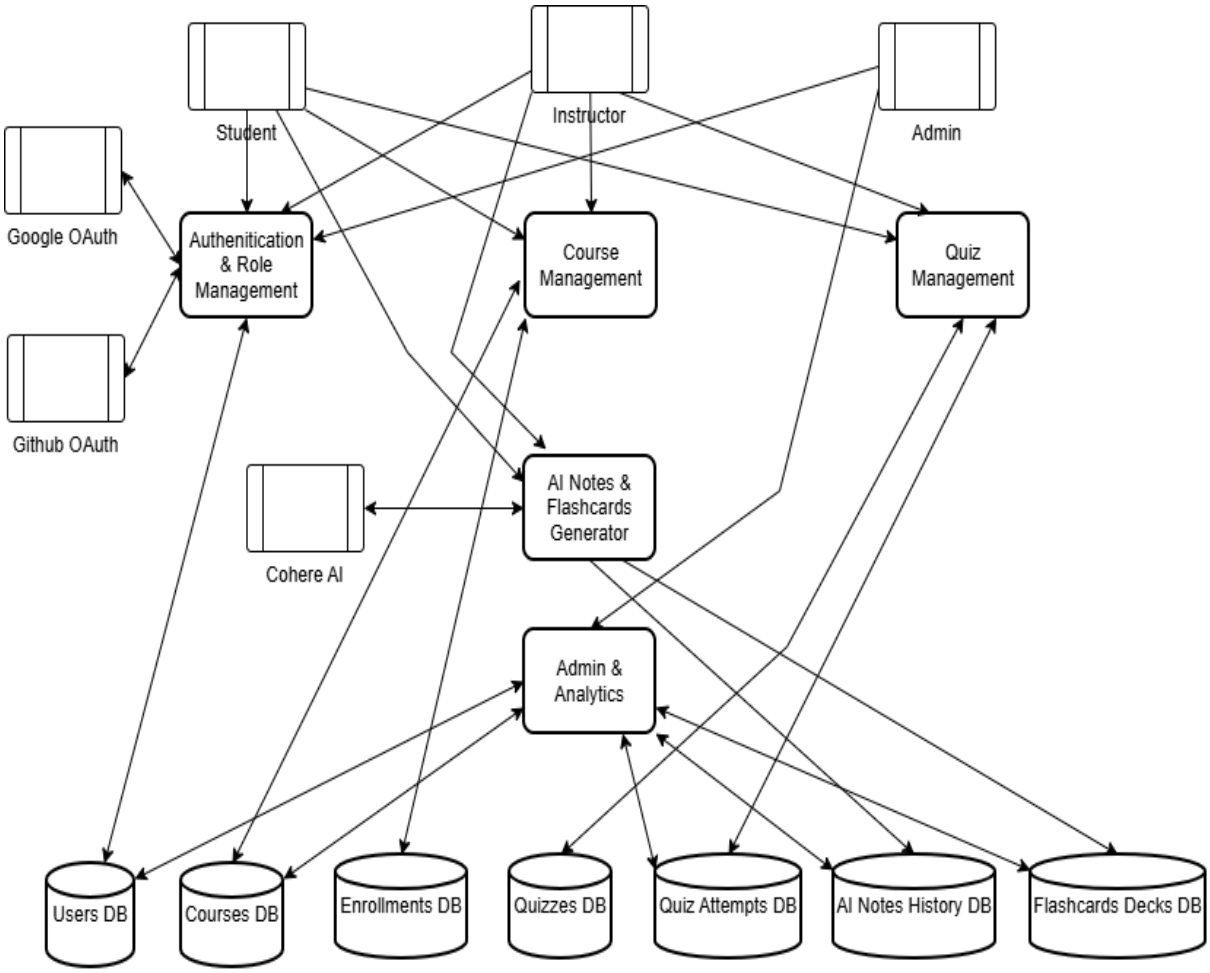


Figure 5.1.2 Level 1 DFD

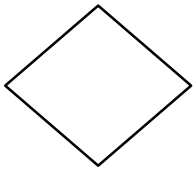
5.2 ENTITY RELATIONSHIP DIAGRAM

An Entity Relationship Diagram (ERD) is a specialized graphic that illustrates the relationship between entities in a database. It is a data modeling technique that graphically illustrates an information system's entities and the relationships between those entities.

An ERD is a conceptual and representational model of data used to represent the entity framework infrastructure.

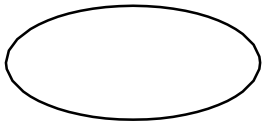
The elements of an ERD are:

- Entities
- Relationship
- Attributes



- The diamond is used to represent relationships

- The box is used to represent the entities



- The oval is used to represent the attributes

- The line is used to represent the flow

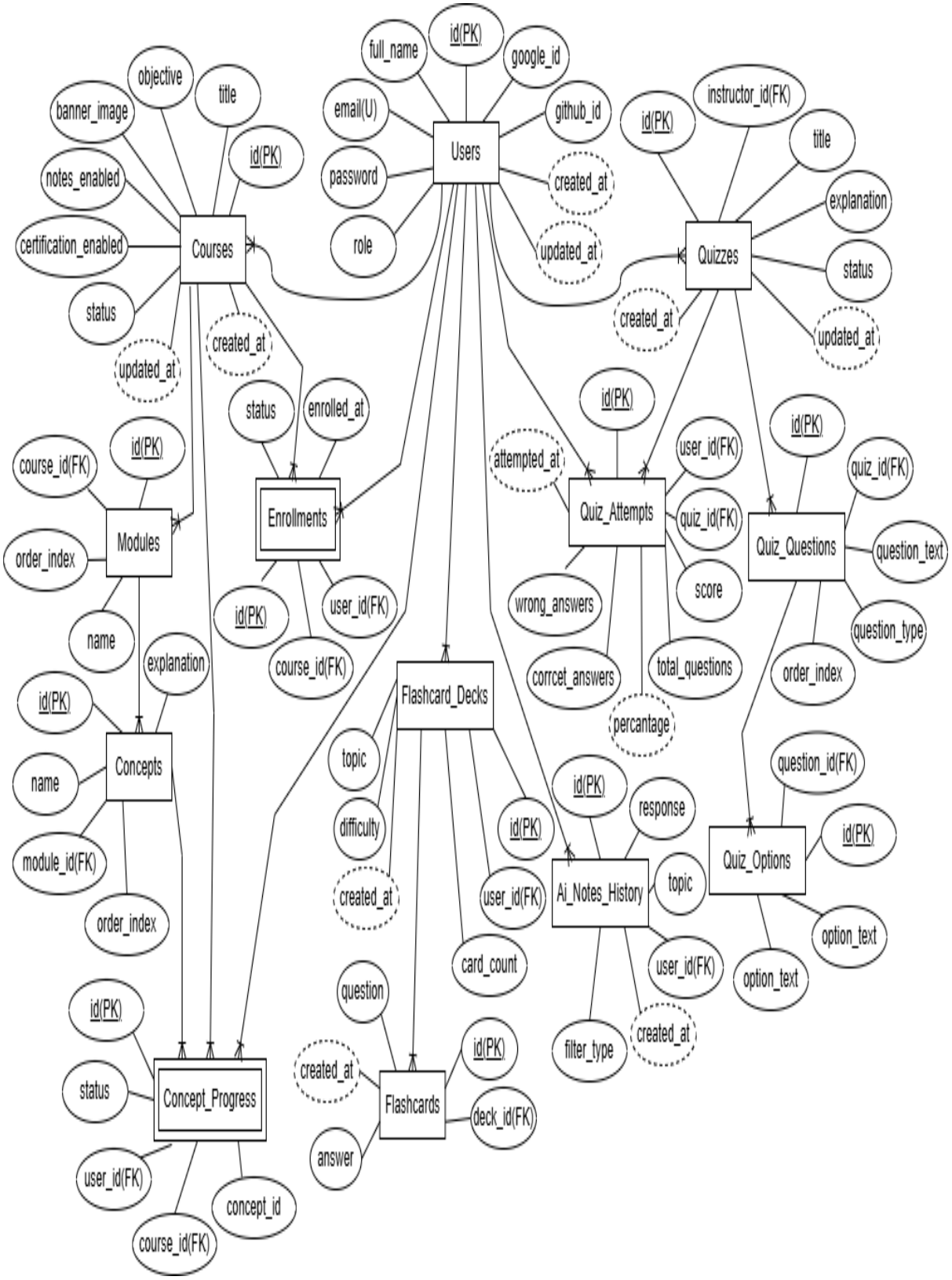


Figure 5.2.1 Entity Relationship Diagram

5.3 INPUT DESIGN

The input design of AcademIQ focuses on creating a structured, user-friendly, and secure method for capturing all necessary information from students, instructors, and administrators. It ensures that each form such as login, signup, course creation, quiz setup, AI note generation, and flashcard creation collects accurate data through validated fields and guided input patterns. The system uses dropdowns, text fields, role selectors, and file upload controls to simplify data entry while minimizing user errors. Proper client-side and server-side validation ensures that inputs such as topics, course modules, questions, and authentication credentials are processed reliably. Overall, the input design guarantees a smooth and accurate data flow into the system, supporting both usability and data integrity.

5.4 OUTPUT DESIGN

The output design of AcademIQ delivers clear, interactive, and visually organized results to users through dashboards, analytics charts, course views, AI-generated notes, flashcards, quiz results, and progress tracking interfaces. Information is presented in a structured layout using cards, tables and formatted content to help users easily understand the generated outputs. Students view course progress, notes, flashcards, certificates, and quiz performance, while instructors receive detailed insights on course engagement and quiz response patterns. SuperAdmins access platform-wide statistics, user growth visuals, and management summaries for efficient decision-making. This output design ensures that system responses are meaningful, accessible, and actionable for every role in the application

CHAPTER 6

SYSTEM IMPLEMENTATION AND TESTING

6.1 SYSTEM IMPLEMENTATION

The implementation of AcademIQ involved building a complete AI-powered Learning Management System using a structured and modular development approach. The frontend was developed using React.js and Tailwind CSS, enabling dynamic page rendering, smooth navigation, and consistent UI components across dashboards, course interfaces, settings pages, quizzes, AI tools, and analytics screens. Every user interaction from login to course access was mapped into responsive components to ensure a seamless experience on different devices.

On the backend, Node.js and Express.js were used to build secure REST APIs responsible for authentication, role verification, course and quiz operations, enrollment handling, progress tracking, and AI services. PostgreSQL was chosen as the database because of its relational structure and capability to efficiently manage complex entities like users, courses, modules, concepts, enrollments, quiz attempts, AI notes, and flashcards. Proper relational mapping and optimized queries ensured fast data retrieval, even with multiple interconnected tables.

AI integration was implemented using Cohere's Command-A model, allowing the system to generate notes, summaries, and flashcards dynamically. OAuth login with Google and GitHub was added to simplify user onboarding and enhance security. Each module Student, Instructor, and SuperAdmin was implemented with role-based route protection, ensuring that users only access features intended for them. Rigorous testing was carried out during each stage of implementation to confirm that screens rendered correctly, APIs communicated without failure, and database operations executed reliably.

6.2 SOFTWARE TESTING

Testing is a process, which reveals errors in the program. It is the major quality measure employed during software development. During testing, the program is executed with a set of conditions known as test cases, and the output is evaluated to determine whether the program is performing as expected.

The process of testing the functionality and correctness of software by running it is called software testing. It is the process of executing a program with the intent of finding an error. A good test case has a high probability of finding a yet undiscovered error. A successful test uncovers a yet undiscovered error. Software testing is usually performed for two reasons.

- Defect detection
- Reliability estimation.

6.2.1 Functional Testing

Functional testing was performed to verify that every feature works according to the system requirements. All major modules authentication, course creation, enrollment, quiz creation, quiz attempts, AI note generation, flashcard generation, and admin features were executed and validated. Each screen and API were tested to ensure accurate input processing and correct output generation. This guaranteed that all user actions functioned smoothly without errors.

6.2.2 Integration Testing

Integration testing ensured proper communication between the frontend, backend, database, and external AI/OAuth services. React components were tested against Express APIs to confirm that data was transmitted and received correctly. Database interactions, such as inserting courses, retrieving quizzes, and updating progress, were validated thoroughly. This testing helped confirm that combined modules worked as a unified system.

6.2.3 User Interface (UI) Testing

User interface testing checked the consistency, responsiveness, and clarity of all screens such as dashboards, course pages, AI tools, and quiz interfaces. The UI was validated across different screen sizes to ensure that layouts, buttons, forms, and navigation elements displayed properly. This testing confirmed that the design was intuitive and user-friendly for Students, Instructors, and SuperAdmins.

6.2.4 Security Testing

Security testing focused on access protection, token validation, and role-based restrictions. Routes were tested to ensure unauthorized users could not access instructor or admin pages. Input validation was checked to prevent SQL injection and invalid data submission. OAuth authentication and JWT tokens were tested to confirm secure handling of user sessions.

6.2.5 User Acceptance Testing (UAT)

User acceptance testing involved navigating through the entire system like a real user to validate usability and end-to-end functionality. All workflows login, browsing courses, creating content, generating notes, taking quizzes, and viewing analytics were executed in real conditions. This testing confirmed that the system meets user expectations and provides a smooth, reliable experience.

.

CHAPTER 7

CONCLUSION & FUTURE RECOMMENDATION

7.1 CONCLUSION

AcademIQ demonstrates how modern web development and artificial intelligence can come together to create a practical and effective learning platform. The system integrates a responsive user interface with reliable server side services and a well structured relational database so that every feature works smoothly from signup and authentication to course delivery, quiz management, and analytics. The inclusion of Cohere based assistance for generating study notes and flashcards reduces manual effort for instructors and gives students faster access to quality learning materials. Role based access ensures that each user group receives the right tools and views which makes daily operations simpler for students, instructors, and administrators. Through careful implementation and systematic testing the platform has proven to be stable, secure, and ready for real usage. Functional checks verified that every workflow behaves as intended. Integration checks validated data flow across the client, the server, the database, and external services. Interface checks confirmed that pages are clear and consistent across devices. The overall outcome is a dependable learning management system that improves teaching productivity, supports student progress, and gives administrators clear visibility and control. AcademIQ provides a solid foundation for future extensions such as richer analytics, more adaptive learning paths, and deeper content automation.

7.2 FUTURE RECOMMENDATION

- **Adaptive Learning and Personalized Recommendations**

The system can be enhanced with adaptive learning algorithms that analyze student performance and automatically recommend suitable courses, modules, or difficulty levels. This would help personalize the learning experience and guide students through the most effective learning paths.

- **Real-Time Virtual Classrooms and Live Sessions**

A live teaching module can be added where instructors conduct real-time classes using video streaming and screen sharing. Integrating attendance tracking, live chat, and interactive whiteboards would further strengthen the learning environment.

- **Advanced Analytics and Predictive Insights**

Future versions can include deeper analytics such as prediction of student performance, risk detection for low engagement, and automated insights for instructors and administrators. This would help in making data-driven academic decisions.

- **Mobile Application Development**

Developing dedicated Android and iOS applications will allow learners and instructors to access courses, quizzes, and AI tools more conveniently. Push notifications and offline access can further enhance usability on mobile devices.

- **Gamification of Learning Activities**

Features such as achievement badges, progress milestones, leaderboards, and reward points can be added to improve engagement. Gamification encourages consistent learning and increases motivation among students.

- **Multilingual Support for Global Accessibility**

The system can be expanded by integrating multilingual interfaces and AI-generated notes in different languages. This would enable learners from diverse linguistic backgrounds to use the platform effectively.

- **AI-Generated Video Summaries and Audio Notes**

Future improvements can include AI-powered video summarization and automatic audio note generation. These features will help students quickly review content through short summaries or listen to study material on the go.

BIBLIOGRAPHY

The following books and websites were referred to during the analysis and execution phase of the project.

WEBSITE REFERENCE

- Tailwind Labs. (2024). Tailwind CSS Documentation. Retrieved from https://tailwindcss.com/docs
- Node.js Foundation. (2024). Node.js Documentation. Retrieved from https://nodejs.org/en/docs
- Express.js. (2024). Express – Fast, unopinionated, minimalist web framework for Node.js. Retrieved from https://expressjs.com/
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. Retrieved from https://www.postgresql.org/docs/
- Cohere. (2025). Cohere API Reference – Command family. Retrieved from https://docs.cohere.com/
- Google Developers. (2024). Google Identity: OAuth 2.0 for Web Server Applications. Retrieved from https://developers.google.com/identity/protocols/oauth2
- GitHub Docs. (2024). Authorizing OAuth Apps. Retrieved from https://docs.github.com/en/apps/oauth-apps/building-oauth-apps/authorizing-oauth-apps
- Chart.js. (2024). Chart.js Documentation. Retrieved from https://www.chartjs.org/docs/latest/
- Neon. (2024). Neon Documentation — Serverless Postgres. Retrieved from https://neon.tech/docs

APPENDIX

SCREENSHOTS

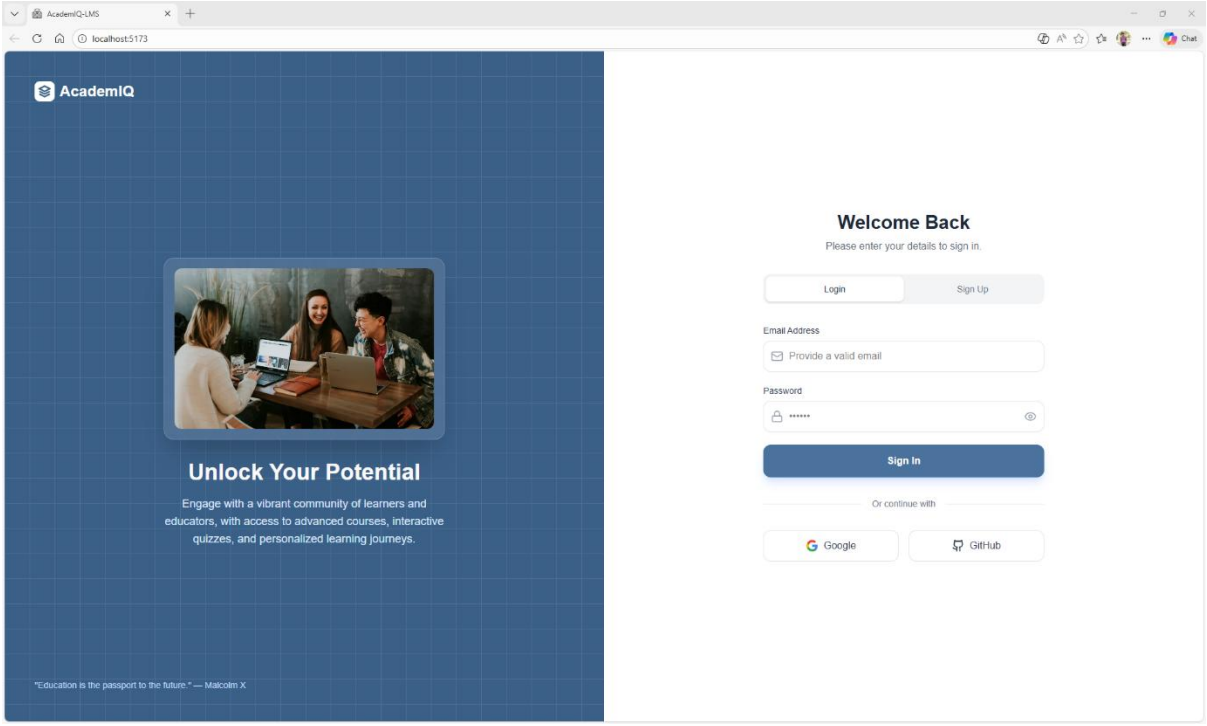


Figure. Login Page

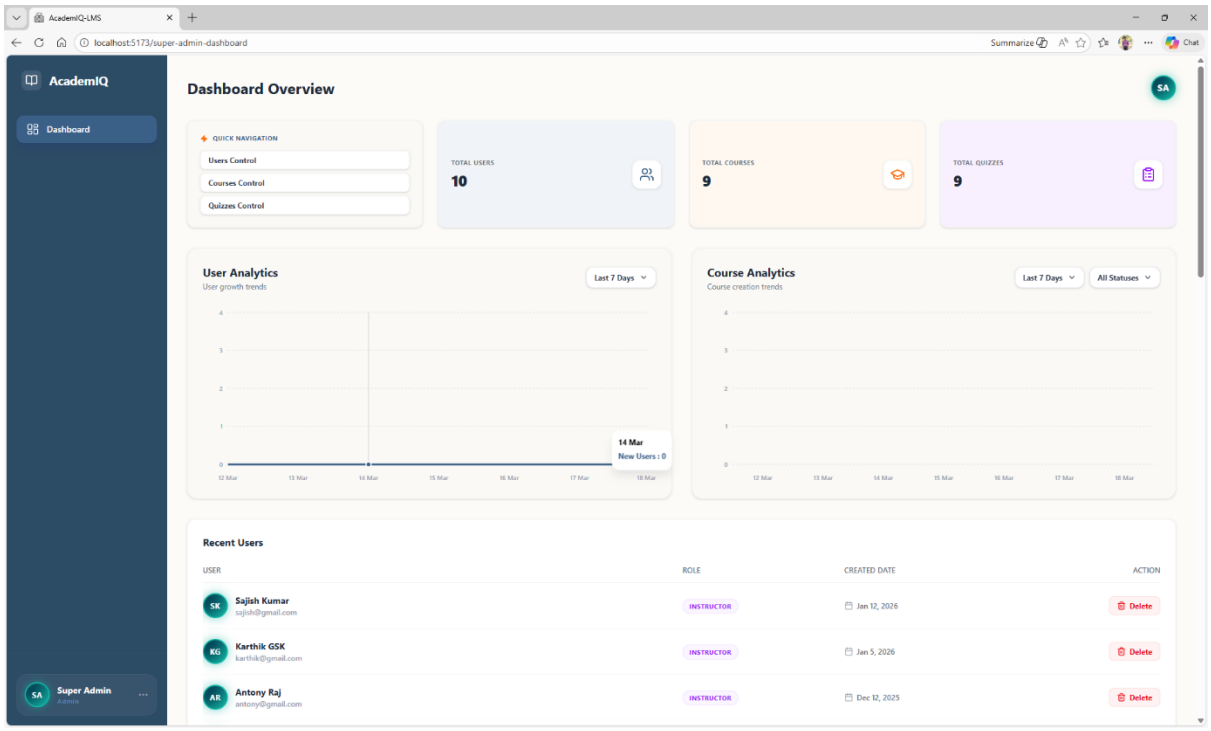


Figure. Admin Control Page

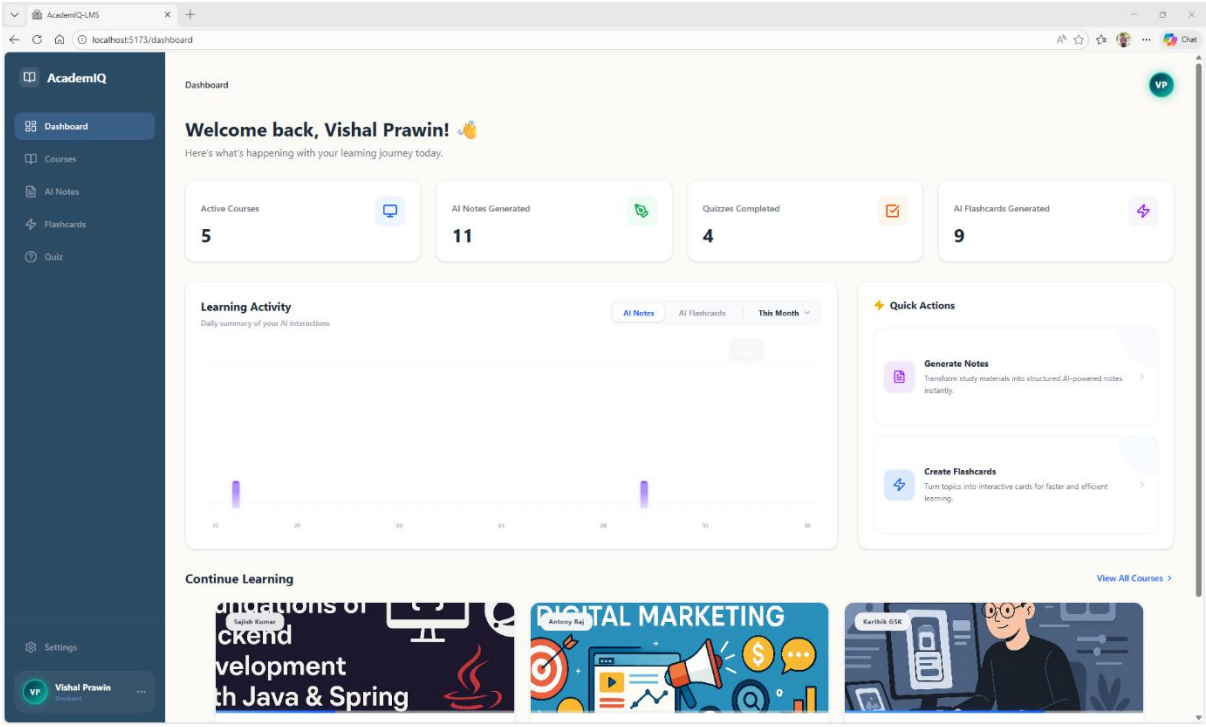


Figure. Dashboard Page

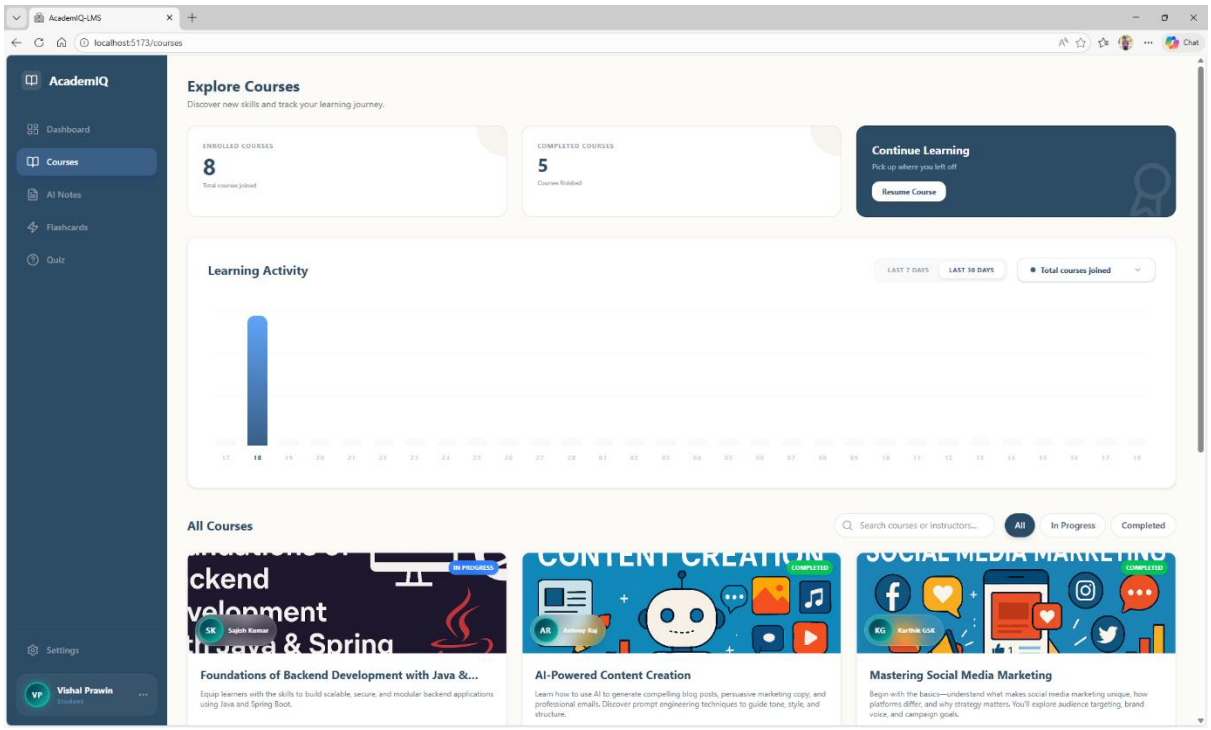


Figure. Courses Page

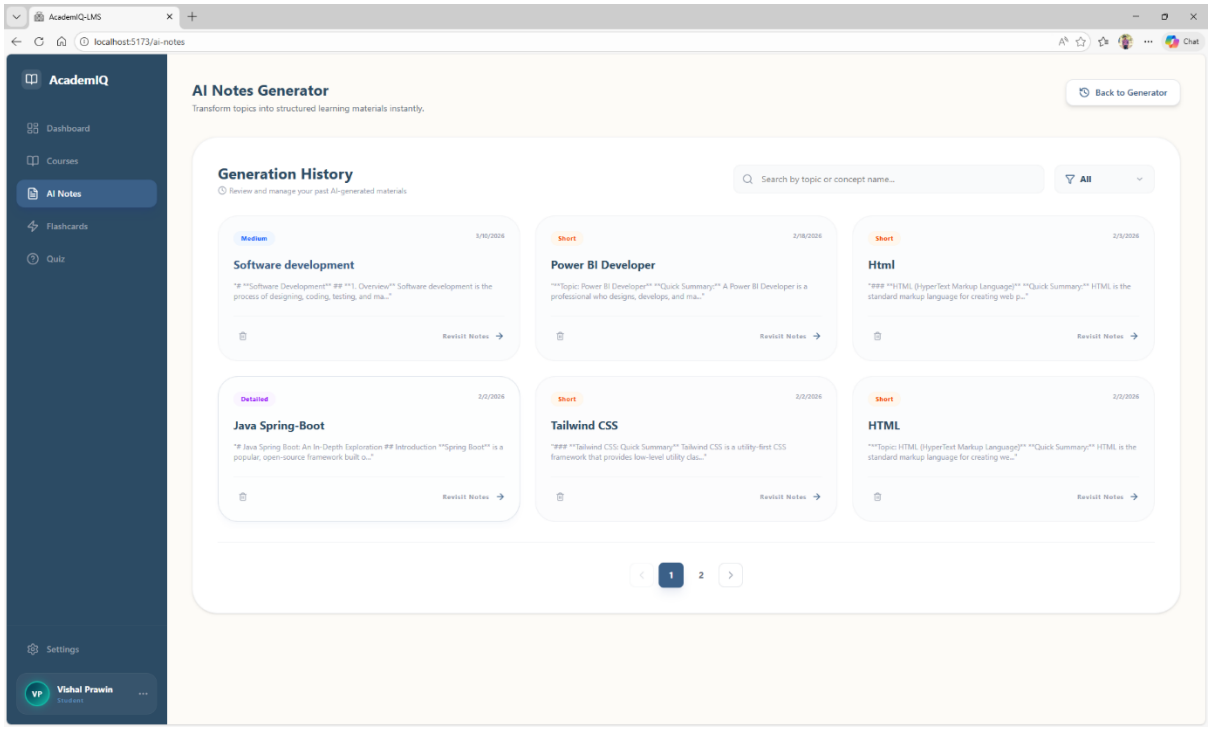


Figure. AI Notes Generation Page

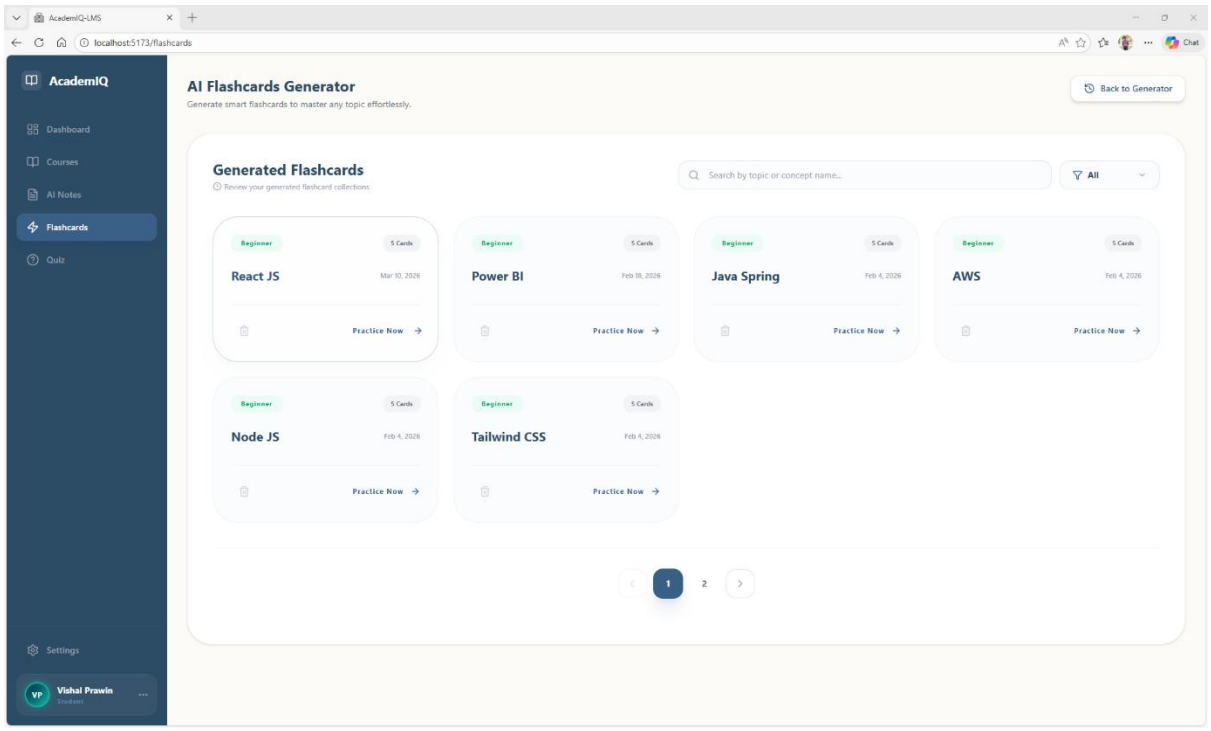


Figure. AI Flashcards Page

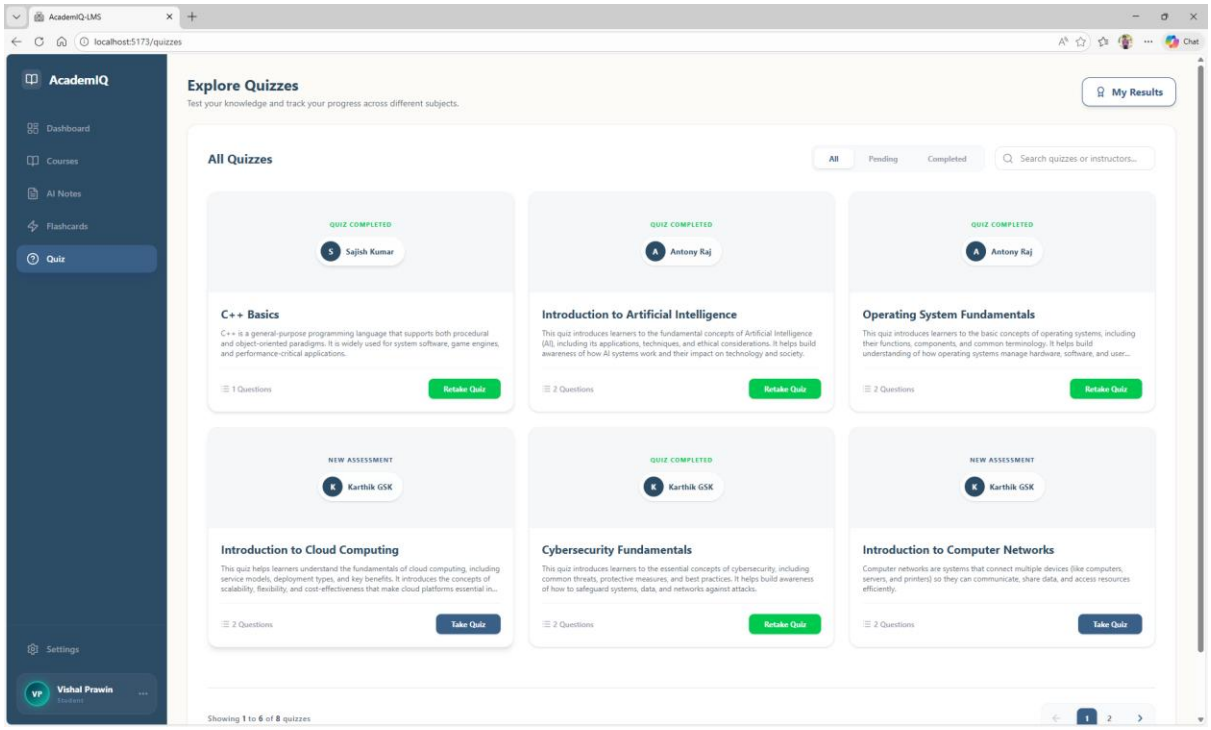


Figure. Quizzes Page

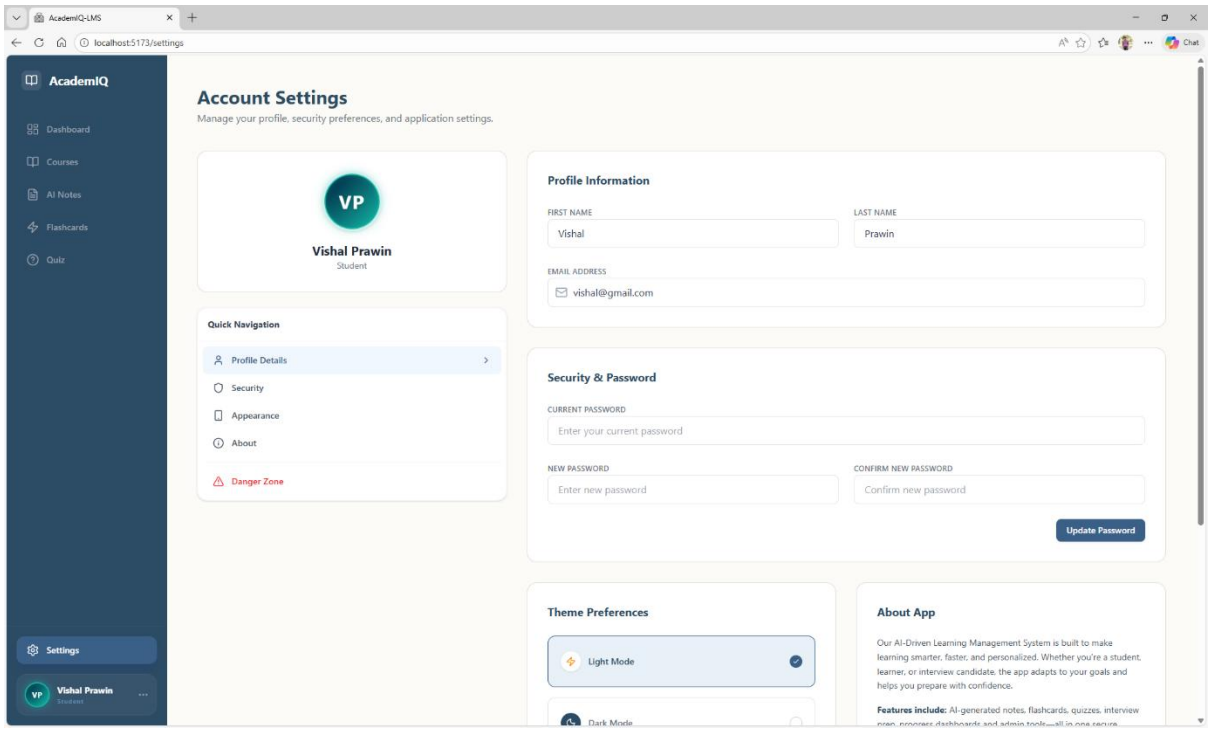


Figure. Settings Page

SAMPLE CODE:

```
import React, { useEffect } from 'react';
import { useNavigate, useSearchParams } from 'react-router-dom';

const AuthCallback = () => {
   const [searchParams] = useSearchParams();
   const navigate = useNavigate();

   useEffect(() => {
      const token = searchParams.get('token');
      const userStr = searchParams.get('user');

      console.log('AuthCallback - Token received:', token ? 'Yes' : 'No');
      console.log('AuthCallback - User string:', userStr);

      if (token && userStr) {
         try {
            const user = JSON.parse(decodeURIComponent(userStr));
            console.log('AuthCallback - Parsed user data:', user);
            console.log('AuthCallback - Avatar URL:', user.avatar);

            localStorage.setItem('token', token);
            localStorage.setItem('user', JSON.stringify(user));

            console.log('AuthCallback - Data saved to localStorage');

            // Redirect to login page with success flag
            navigate('/?login_success=true');

            // You might want to trigger a toast here via a global context or event
            // But since we are redirecting, the destination page would need to handle
"Welcome back"
         } catch (error) {
            console.error('Failed to parse user data', error);
            navigate('/?error=auth_failed');
         }
      } else {
         navigate('/?error=no_token');
      }
   }, [searchParams, navigate]);

   return (
      <div className="min-h-screen flex items-center justify-center bg-gray-50">
         <div className="text-center">
            <div className="w-16 h-16 border-4 border-blue-600 border-t-transparent
rounded-full animate-spin mx-auto mb-4"></div>
            <h2 className="text-xl font-semibold text-gray-700">Completing
authentication...</h2>
         </div>
```

```
        </div>
    );
};

export default AuthCallback;

const Dashboard = () => {
    const navigate = useNavigate();
    const [user, setUser] = useState({
        fullName: 'Guest User',
        email: '',
        avatar: null
    });
    const [showProfileMenu, setShowProfileMenu] = useState(false);
    const [showSidebarProfileMenu, setShowSidebarProfileMenu] = useState(false);
    const [headerSearch, setHeaderSearch] = useState('');
    const [stats, setStats] = useState({
        completedCourses: 0,
        aiNotesGenerated: 0,
        quizAttempts: 0,
        flashcardsGenerated: 0
    });
    const [loadingStats, setLoadingStats] = useState(false);
    const [inProgressCourses, setInProgressCourses] = useState([]);
    const [instructorCourses, setInstructorCourses] = useState([]);
    const [carouselOffset, setCarouselOffset] = useState(0);
    const [activeActivityType, setActiveActivityType] = useState('notes'); // 'notes' or
'flashcards'
    const [activeActivityRange, setActiveActivityRange] = useState(7); // 7 or 30
    const [activityChartData, setActivityChartData] = useState([]);
    const [showRangeDropdown, setShowRangeDropdown] = useState(false);
    const profileMenuRef = useRef(null);
    const sidebarProfileRef = useRef(null);
    const rangeDropdownRef = useRef(null);
    const activePage = 'Dashboard';

    const handleHeaderSearch = (e) => {
        if (e.key === 'Enter' && headerSearch.trim()) {
            navigate(`/courses?search=${encodeURIComponent(headerSearch.trim())}`);
        }
    };

    useEffect(() => {
        const storedUser = localStorage.getItem('user');
        let currentUser = null;
        if (storedUser) {
            try {
                const parsedUser = JSON.parse(storedUser);
                setUser(parsedUser);
                currentUser = parsedUser;
```

```
        } catch (e) {
            console.error('Failed to parse user data');
        }
    }

    if (currentUser) {
        const role = currentUser.role?.toLowerCase();
        fetchStats(currentUser.id, role);
        if (role === 'student') {
            fetchInProgressCourses(currentUser.id);
            fetchLearningActivity(currentUser.id, activeActivityType, activeActivityRange,
'student');
        } else if (role === 'instructor') {
            fetchInstructorCourses(currentUser.id);
            // Set default type to 'courses' for instructor if it's currently a student type
            if (activeActivityType === 'notes' || activeActivityType === 'flashcards') {
                setActiveActivityType('courses');
            }
            fetchLearningActivity(currentUser.id, activeActivityType === 'notes' ||
activeActivityType === 'flashcards' ? 'courses' : activeActivityType, activeActivityRange,
'instructor');
        }
    }

    const handleClickOutside = (event) => {
        if (profileMenuRef.current && !profileMenuRef.current.contains(event.target)) {
            setShowProfileMenu(false);
        }
        if (sidebarProfileRef.current && !sidebarProfileRef.current.contains(event.target)) {
            setShowSidebarProfileMenu(false);
        }
        if (rangeDropdownRef.current &&
!rangeDropdownRef.current.contains(event.target)) {
            setShowRangeDropdown(false);
        }
    };

    document.addEventListener('mousedown', handleClickOutside);
    return () => {
        document.removeEventListener('mousedown', handleClickOutside);
    };
  }, []);
const [userGrowthData, setUserGrowthData] = useState([]);
  const [userGrowthDays, setUserGrowthDays] = useState(7);
  const [loadingUserGrowth, setLoadingUserGrowth] = useState(true);
  const [showUserGrowthFilter, setShowUserGrowthFilter] = useState(false);

  const [courseGrowthData, setCourseGrowthData] = useState([]);
  const [courseGrowthDays, setCourseGrowthDays] = useState(7);
  const [courseGrowthStatus, setCourseGrowthStatus] = useState('all');
```

```
  const [loadingCourseGrowth, setLoadingCourseGrowth] = useState(true);
  const [showCourseDaysFilter, setShowCourseDaysFilter] = useState(false);
  const [showCourseStatusChartFilter, setShowCourseStatusChartFilter] = useState(false);

  const profileMenuRef = useRef(null);
  const recentUsersRef = useRef(null);
  const courseManagementRef = useRef(null);
  const quizManagementRef = useRef(null);
  const sidebarProfileRef = useRef(null);

  useEffect(() => {
    fetchUsers();
    fetchCourses();
    fetchQuizzes();
    fetchUserGrowth();
    fetchCourseGrowth();
  }, []);

  useEffect(() => {
    fetchUserGrowth();
  }, [userGrowthDays]);

  useEffect(() => {
    fetchCourseGrowth();
  }, [courseGrowthDays, courseGrowthStatus]);

  const fetchUserGrowth = async () => {
    setLoadingUserGrowth(true);
    try {
      const response = await fetch(`http://localhost:5000/api/analytics/user-
growth?days=${userGrowthDays}`);
      if (response.ok) {
        const data = await response.json();
        setUserGrowthData(data);
      }
    } catch (error) {
      console.error('Error fetching user growth data:', error);
    } finally {
      setLoadingUserGrowth(false);
    }
  };

  const fetchCourseGrowth = async () => {
    setLoadingCourseGrowth(true);
    try {
      const response = await fetch(`http://localhost:5000/api/analytics/course-
growth?days=${courseGrowthDays}&status=${courseGrowthStatus}`);
      if (response.ok) {
        const data = await response.json();
        setCourseGrowthData(data);
```

```
        }
    } catch (error) {
        console.error('Error fetching course growth data:', error);
    } finally {
        setLoadingCourseGrowth(false);
    }
};

const showToast = (message, type = 'success') => {
    setToast({ message, type });
};

const fetchCourses = async () => {
    try {
        const response = await fetch('http://localhost:5000/api/courses');
        if (response.ok) {
            const data = await response.json();
            // Handle both array and object responses
            const coursesList = Array.isArray(data) ? data : (data.courses || []);
            setCourses(coursesList);
            setStats(prev => ({ ...prev, activeCourses: coursesList.length }));
        }
    } catch (error) {
        console.error('Error fetching courses:', error);
        setCourses([]);
    } finally {
        setLoadingCourses(false);
    }
};
-- Create users table
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    full_name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255),
    google_id VARCHAR(255) UNIQUE,
    github_id VARCHAR(255) UNIQUE,
    avatar_url TEXT,
    role VARCHAR(50) DEFAULT 'student',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Create index on email for faster lookups
CREATE INDEX IF NOT EXISTS idx_users_email ON users(email);
const pool = require('./db');

async function setupAINotesTable() {
    try {
        await pool.query(`
```

```
        CREATE TABLE IF NOT EXISTS ai_notes_history (
          id SERIAL PRIMARY KEY,
          user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
          topic TEXT NOT NULL,
          filter_type VARCHAR(50) NOT NULL,
          response TEXT NOT NULL,
          created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
      `);
      console.log('ai_notes_history table created successfully');
      process.exit(0);
    } catch (error) {
      console.error('Error creating ai_notes_history table:', error);
      process.exit(1);
    }
}

setupAINotesTable();
```

SYNOPSIS

AcademIQ is an AI-powered Learning Management System designed to modernize and streamline digital education through intelligent automation and a user-friendly interface. The system integrates various modules such as course management, content delivery, student tracking, and instructor tools into a single platform. It uses modern frontend technologies like React.js and Tailwind CSS to provide a responsive, fast, and seamless user experience suitable for both students and educators.

The platform incorporates secure authentication through Google and GitHub OAuth, along with a role-based access system that categorizes users as Students, Instructors, or SuperAdmins. Each role is provided with dedicated dashboards and permissions that align with its responsibilities, ensuring structured workflow and controlled access. The backend, powered by Node.js, Express.js, and PostgreSQL, manages data efficiently while maintaining scalability, security, and smooth API communication across the system.

A key highlight of AcademIQ is its integration with Cohere AI (Command-A), which enables intelligent features such as AI-assisted content generation, automated summary creation, and personalized learning support. This reduces the manual workload for instructors and enhances the learning experience for students. By combining a powerful technical stack with smart AI capabilities, AcademIQ delivers a next-generation LMS solution that is efficient, interactive, and adaptable to the evolving needs of digital education.

Perumal P (21MSS034), Fifth Year M.Sc. Software Systems,
Batch (2021-2026), PSG College of Arts & Science, Coimbatore